

Verificación de RTL con cocotb

Tutorial práctico: Makefile, testbench y ondas

Curso de FEC 2026 — Fundación Fulgor

¿Qué es cocotb?

cocotb (*coroutine-based cosimulation testbench*) permite escribir el banco de pruebas en **Python** en lugar de SystemVerilog.

El diseño bajo prueba (DUT) se compila y se ejecuta en un simulador HDL convencional. cocotb se conecta a ese simulador mediante su interfaz de programación (VPI en Verilog) y desde allí manipula las señales.

Consecuencias prácticas

- ▶ No se escribe HDL de testbench: no hay `initial`, ni tareas, ni clases UVM.
- ▶ Se dispone de todo el ecosistema de Python: NumPy, modelos de referencia propios, archivos, gráficos.
- ▶ El RTL **no se modifica**: cocotb no es un lenguaje de descripción de hardware.

En el curso lo utilizamos para contrastar el RTL contra un modelo de referencia escrito con las clases de álgebra desarrolladas en el TP1.

Requisitos para ejecutar los ejemplos

Componente	Función
Python 3	Intérprete donde corre el testbench
cocotb	Paquete de Python (<code>pip install cocotb</code>)
Simulador HDL	Ejecuta el RTL. En el curso: Icarus Verilog
GNU make	Orquesta compilación y simulación
GTKWave	Visualización de ondas (opcional)

El paquete cocotb instala el ejecutable cocotb-config, que el Makefile utiliza para localizar sus archivos internos. Debe estar accesible en el PATH:

```
$ cocotb-config --makefiles
$ iverilog -V
```

Si se trabaja dentro de un entorno virtual, éste debe estar activado en la misma terminal desde la que se invoca make.

Estructura del proyecto

cocotb no impone una organización de directorios. La siguiente resulta cómoda cuando varios módulos comparten un mismo modelo de referencia:

```
curso_fec/  
|-- algebra/  
|   |-- galois.py           <- modelo de referencia compartido  
|-- unidad3/  
    |-- hamming_7_4_encoder/  
        |-- Makefile  
        |-- hamming_encoder.sv      <- DUT  
        |-- hamming_encoder_tb.py   <- testbench  
        |-- dump.sv                 <- volcado de ondas
```

Cada directorio de módulo es autónomo: se ingresa a él y se ejecuta `make`. El modelo de referencia se comparte agregando su ruta al `PYTHONPATH` desde el `Makefile`.

El Makefile: variables esenciales

El flujo estándar consiste en definir un conjunto de variables e incluir el Makefile que provee cocotb.

Variable	Significado
SIM	Simulador a utilizar (icarus, verilator, ...)
TOPLEVEL_LANG	Lenguaje del módulo superior (verilog o vhd1)
VERILOG_SOURCES	Lista de archivos fuente a compilar
TOPLEVEL	Nombre del módulo que cocotb toma como DUT
MODULE	Módulo de Python que contiene los tests

```
include $(shell cocotb-config --makefiles)/Makefile.sim
```

MODULE se indica sin la extensión .py. A partir de cocotb 2.0 pasó a llamarse COCOTB_TEST_MODULES; MODULE sigue funcionando, pero emite un aviso de obsolescencia.

El Makefile: ejemplo completo

```
TOPLEVEL_LANG ?= verilog
SIM            ?= icarus

VERILOG_SOURCES := $(PWD)/hamming_encoder.sv $(PWD)/dump.sv
TOPLEVEL        := hamming_encoder
MODULE          := hamming_encoder_tb

# para que el testbench encuentre galois.py
export PYTHONPATH := $(PWD):$(PWD)/../../algebra:$(PYTHONPATH)

COCOTB_HDL_TIMEUNIT      = 1ns
COCOTB_HDL_TIMEPRECISION = 1ps

ifeq ($(SIM),icarus)
    COMPILE_ARGS += -g2012 -s dump
endif

include $(shell cocotb-config --makefiles)/Makefile.sim

.PHONY: ondas
ondas: sim
    gtkwave hamming_encoder.vcd &
```

-g2012 habilita SystemVerilog en Icarus. Los *timeunits* son necesarios cuando el RTL no declara `'timescale`.

El testbench mínimo

Un test es una corrutina decorada con `@cocotb.test()`. `cocotb` la descubre automáticamente y le pasa un único argumento: el *handle* del módulo superior.

```
import cocotb
from cocotb.triggers import Timer

@cocotb.test()
async def test_encoder(dut):
    dut.i_info.value = 0b1011
    await Timer(2, "ns")
    assert int(dut.o_message.value) == 0b0001011
```

- ▶ El test finaliza correctamente si la corrutina retorna sin excepciones.
- ▶ Cualquier excepción (típicamente un `AssertionError`) lo marca como fallido.
- ▶ Un mismo archivo puede contener varios tests; se ejecutan secuencialmente.

Puertos: escritura y lectura

El objeto `dut` expone cada puerto como atributo. La escritura y la lectura se realizan siempre a través de `.value`.

```
dut.i_info.value = 11          # decimal
dut.i_info.value = 0b1011     # binario
dut.i_rst.value  = 1

v = int(dut.o_message.value)  # entero
b = str(dut.o_message.value)  # cadena de bits: '0001011'
```

La escritura no es inmediata

Asignar `.value` **programa** el cambio; éste se aplica en el siguiente paso de simulación. Leer la salida inmediatamente después de escribir la entrada devuelve el valor anterior: siempre debe mediar un `await`.

Si la señal contiene valores `x` o `z`, `int()` lanza una excepción. Para diagnosticar conviene inspeccionar `str()`, que los muestra explícitamente.

Control del tiempo: *triggers*

El tiempo de simulación sólo avanza cuando el test cede el control con `await` sobre un *trigger*.

Trigger	Espera hasta
<code>Timer(n, "ns")</code>	que transcurran n unidades de tiempo
<code>RisingEdge(sig)</code>	el próximo flanco ascendente de <code>sig</code>
<code>FallingEdge(sig)</code>	el próximo flanco descendente
<code>Edge(sig)</code>	cualquier cambio de <code>sig</code>
<code>ClockCycles(sig, n)</code>	n flancos de <code>sig</code>

```
from cocotb.triggers import RisingEdge, Timer

await Timer(10, "ns")
await RisingEdge(dut.i_clk)
```

Un test sin ningún `await` se ejecuta enteramente en el instante cero y el DUT nunca llega a evaluarse.

Diseños combinacionales

Un módulo puramente combinacional no requiere reloj. Basta con aplicar el estímulo y conceder un intervalo de propagación antes de muestrear.

```
@cocotb.test()
async def test_hamming_encoder(dut):
    for u in range(16):
        # el encoder es combinacional: aplicamos y esperamos que propague
        dut.i_info.value = u
        await Timer(2, "ns")
        v = int(dut.o_message.value)
        assert v == modelo(u), f"u={u:04b}: DUT={v:07b}, modelo={modelo(u):07b}"
```

El valor concreto del Timer es indiferente mientras sea mayor que cero: el objetivo es únicamente permitir que el simulador propague el cambio.

Generación de reloj

La clase `Clock` genera la señal; `start_soon` la lanza como corrutina concurrente que continúa ejecutándose durante todo el test.

```
from cocotb.clock import Clock

@cocotb.test()
async def test_decoder(dut):
    cocotb.start_soon(Clock(dut.i_clk, 10, "ns").start())
    ...
```

- ▶ El período se indica completo (10 ns), no el semiperíodo.
- ▶ `start_soon` **no** bloquea: la ejecución continúa en la línea siguiente.
- ▶ Sirve también para lanzar monitores o generadores de estímulo en paralelo.

Omitir `start_soon` es un error frecuente: sin reloj, cualquier `await RisingEdge(dut.i_clk)` queda esperando indefinidamente y la simulación se detiene sin avanzar.

Reset y secuencia de arranque

```
# creamos un clock de periodo 10ns  
cocotb.start_soon(Clock(dut.i_clk, 10, "ns").start())  
dut.i_rst.value = 1  
dut.i_message.value = 0  
await RisingEdge(dut.i_clk)  
await RisingEdge(dut.i_clk)  
dut.i_rst.value = 0
```

Todas las entradas deben inicializarse antes del primer flanco. Una entrada que nunca se escribe permanece en x y ese valor se propaga por el diseño, produciendo fallas cuya causa resulta difícil de localizar.

Con reset sincrónico, dos flancos son suficientes. Con reset asincrónico conviene además desactivarlo fuera del flanco activo, para no violar el tiempo de recuperación del registro.

Verificación mediante assert

cocotb no aporta una biblioteca propia de aserciones: se emplea el assert de Python. El mensaje asociado es lo que se leerá cuando el test falle, y conviene que identifique el estímulo y contraste ambos valores.

```
assert info == info_esp, f"r={r:07b}: o_info {info:04b}, modelo {info_esp:04b}"
```

```
AssertionError: r=0000100: o_info 0111, modelo 0011  
** TESTS=1 PASS=0 FAIL=1 SKIP=0 **
```

Los especificadores :04b y :07b de las *f-strings* son particularmente útiles en este contexto, dado que exhiben directamente el patrón de bits en discusión.

Acceso a señales internas

El *handle* del DUT permite descender por la jerarquía utilizando los nombres de instancia del RTL. Esto habilita la verificación de nodos internos sin necesidad de exponerlos como puertos.

```
s = int(dut.u_syndrome_calculator.o_syndrome.value)
m = int(dut.message.value)           # señal interna del propio top
```

Criterio de diseño

La interfaz del módulo se define según las necesidades del sistema, no las del testbench. Si se requiere observar una señal interna, se accede por jerarquía; agregar puertos de observación contamina el diseño.

Icarus admite este acceso sin configuración adicional. En Verilator es necesario compilar con `-public-flat-rw` o marcar las señales con `/* verilator public */`.

Modelo de referencia

Comparar contra constantes escritas a mano no escala y reproduce los errores de interpretación del propio diseñador. Resulta preferible describir el comportamiento esperado de forma independiente.

```
# encoder de referencia: la paridad es el resto de dividir por g(x)  
def modelo(u):  
    return u | (bits((poly(u, 4) * GFPoly(F, [1, 1])) % G, 3) << 4)
```

El modelo se deriva de la especificación algebraica — el polinomio generador — mientras que el RTL implementa una red de compuertas XOR. Ambos caminos son independientes, de modo que su coincidencia sobre el espacio completo de entradas constituye evidencia sólida.

Cuando el espacio de entrada es reducido, como aquí, conviene recorrerlo por completo antes que recurrir a estímulos aleatorios.

Latencia y alineación temporal

En un diseño segmentado, la salida observada tras un flanco corresponde a un estímulo aplicado varios ciclos antes. El testbench debe contemplar ese desplazamiento de manera explícita.

```
for k in range(129):
    if k < 128:
        dut.i_message.value = k
        # esperamos un ciclo de clock
        await RisingEdge(dut.i_clk)
        await Timer(1, "ns")

    if k >= 1:
        r = k - 1 # nos desplazamos un ciclo por la latencia del modulo
        assert int(dut.o_info.value) == modelo(r)[2]
```

El `Timer(1, "ns")` posterior al flanco evita muestrear en el instante exacto de la transición. Obsérvese que el bucle realiza una iteración adicional, destinada a drenar el último dato del *pipeline*.

Volcado de ondas

El volcado se solicita desde HDL. En lugar de modificar el RTL, se agrega un módulo independiente dedicado exclusivamente a esa tarea:

```
module dump;

initial begin
    $dumpfile("hamming_encoder.vcd");
    $dumpvars(0, hamming_encoder);
end

endmodule
```

- ▶ `$dumpvars(0, ...)` incluye todos los niveles de jerarquía por debajo del módulo indicado.
- ▶ El módulo no instancia nada: referencia al DUT por su nombre de instancia raíz.
- ▶ Por tratarse de una segunda raíz, Icarus debe elaborarla explícitamente: `COMPILE_ARGS += -s dump`.

Ejecución y resultados

```
$ make           # compila y simula
$ make ondas     # simula y abre GTKWave
$ make clean     # elimina sim_build
```

```
0.00ns INFO cocotb.regression running test_hamming_encoder
** TEST                STATUS  SIM TIME  RATIO **
** test_hamming_encoder  PASS    33.00ns   8905 **
** TESTS=1 PASS=1 FAIL=0 SKIP=0                        **
```

La ejecución genera además `results.xml` en formato JUnit, apto para su integración en un sistema de integración continua. Todo lo impreso con `print()` desde el testbench aparece intercalado en la salida, lo que permite emitir tablas de resultados junto con la verificación.

Dificultades frecuentes

Síntoma	Causa habitual
<code>cocotb-config: command not found</code>	Entorno virtual sin activar, o PATH incompleto
<code>syntax error</code> en Icarus	Falta <code>-g2012</code> para SystemVerilog
La simulación no avanza	Reloj sin lanzar con <code>start_soon</code>
Se leen valores previos	Falta un <code>await</code> entre escritura y lectura
Aparecen valores <code>x</code>	Entradas sin inicializar antes del primer flanco
Los cambios del RTL no surten efecto	<code>sim_build</code> obsoleto: ejecutar <code>make clean</code>

El último caso se presenta con particular frecuencia bajo WSL cuando el proyecto reside en `/mnt/c`, dado que las marcas temporales del sistema de archivos de Windows no siempre resultan confiables para `make`.

Resumen

Flujo de trabajo

1. Declarar fuentes, TOPLEVEL y MODULE en el Makefile.
2. Escribir el modelo de referencia a partir de la especificación.
3. Recorrer el espacio de entrada y contrastar DUT contra modelo.
4. Ante una falla, examinar el mensaje del assert y luego las ondas.

Referencias:

- ▶ Documentación oficial: docs.cocotb.org
- ▶ Icarus Verilog: steveicarus.github.io/iverilog
- ▶ GTKWave: gtkwave.sourceforge.net